

Technical Architecture & Flow Guide: Strata Soil Recommendation System

This document provides a comprehensive walkthrough of the Strata system architecture, detailing the end-to-end execution flow from physical sensor scanning up to app integration and machine learning inference.

1. System Overview & Component Stack

Strata is an IoT-enabled soil analyzer powered by a Raspberry Pi running an asynchronous Python service (`ble_server.py`). The system is integrated with the following components:

- **7-in-1 Soil Sensor:** Reads moisture, temperature, EC, pH, Nitrogen (N), Phosphorus (P), and Potassium (K) via RS485 Modbus RTU on `/dev/ttyUSB0`.
 - **Nextion HMI Touch Display:** Connected via UART (`/dev/ttyAMA0`) to show dashboards, scanning state, and raw results.
 - **Physical Scan Button:** A tactile button mapped to GPIO11 to trigger offline scans.
 - **Buzzer:** Connected to GPIO17 to provide audio cues (scan starts, scan ticks, scan finished, and errors).
 - **Waveshare UPS HAT (E):** Monitored via the I2C bus (`0x2d`) to track battery capacity.
 - **Machine Learning Model:** Loaded from `STRATA_FINAL_FINAL3.pkl` containing a pre-trained Random Forest Classifier, label encoders, and parameter thresholds.
 - **Local DB:** SQLite database (`raw_scans.db`) that persists historical scan logs.
-

2. Power System & Battery Lifespan (Waveshare UPS HAT E)

The Strata system is designed for portable, battery-powered operation. It utilizes the **Waveshare UPS HAT (E)** for power delivery and monitoring.

Battery Specifications

- **Cell Configuration:** 4 × 21700 lithium-ion batteries connected in series.
- **Nominal Chemistry:** 3.7V per cell (4.2V fully charged, 3.0V safe discharge cutoff limit).
- **Pack Voltage (Nominal):** $4 \times 3.7\text{V} = 14.8\text{V}$ (maximum charge voltage $\approx 16.8\text{V}$).
- **Total Pack Energy (Wh):**
- **With 5000 mAh (5.0 Ah) cells:** $14.8\text{V} \times 5.0\text{Ah} = 74.0\text{Wh}$ (standard high-capacity 21700 batteries).

- With 4000 mAh (4.0 Ah) cells: $14.8\text{V} \times 4.0\text{Ah} = 59.2\text{Wh}$.

Power Consumption & Runtime Estimations

Power draw fluctuates based on CPU activity, wireless transmission state, display brightness, and sensor usage:

Subsystem	Power Consumption (W)	Description
Raspberry Pi 4 / 5	3.5W – 6.5W	Draws $\approx 3.5\text{W}$ at idle/background, spiking to $\approx 6.5\text{W}$ during Modbus RTU scanning and ML inference.
Nextion HMI Display	1.0W – 1.5W	Depends on screen backlight level (set at full brightness/ <code>color="#c7254e" size="9">dim=100</code>).</code>
Soil Sensor & Peripherals	0.3W – 0.5W	Continuous Modbus polling, buzzer trigger, and indicator lines.
UPS DC-DC Converter Loss	0.8W – 1.2W	The UPS HAT operates at $\approx 85\%$ efficiency, introducing a 15% conversion loss.
Total System Load (Draw)	5.6W – 9.7W	Typical average load: $\approx 6.8\text{W}$. Peak load (scanning/computing): $\approx 9.7\text{W}$.

Based on these consumption characteristics, the estimated runtimes are:

- **Continuous Active Operation (Heavy Scanning/Inference):**
 - With 5000 mAh batteries: $\frac{74\text{Wh}}{9.7\text{W}} \approx \mathbf{7.6\text{ hours}}$
 - With 4000 mAh batteries: $\frac{59.2\text{Wh}}{9.7\text{W}} \approx \mathbf{6.1\text{ hours}}$
- **Standard Idle/Standby Operation (Dashboard View):**
 - With 5000 mAh batteries: $\frac{74\text{Wh}}{5.6\text{W}} \approx \mathbf{13.2\text{ hours}}$
 - With 4000 mAh batteries: $\frac{59.2\text{Wh}}{5.6\text{W}} \approx \mathbf{10.5\text{ hours}}$

Low Battery Warning & Safe Shutdown

- **Cutoff Threshold:** Lithium-ion cells degrade rapidly if discharged below 3.0V. A **"Low Battery"** warning is configured when the pack drops to **20% state of charge (SoC)** (equivalent to $\approx 13.0\text{V}$ - 13.2V remaining).
- **Time to reach Low Battery State:** It takes approximately **6.0 to 10.5 hours** of active standby/scanning to reach the 20% remaining capacity warning state.
- **Protection Action:** The system queries the onboard battery fuel gauge IC via the I2C bus (`0x2d` register `0x20`) every 2 seconds. When the capacity falls below the critical threshold, a safe shutdown command (`sudo shutdown -h now`) is invoked to protect the cells.

3. BLE Connection & Communication Architecture

Strata operates a Bluetooth Low Energy (BLE) peripheral server utilizing the `ble` library. It advertises the main service and maintains characteristics that the mobile application subscribes to or writes to.

GATT Service and Characteristics

- **Service Name:** `Strata`
- **Service UUID:** `56c36f56-da27-464a-952a-9e6631168f6d`

Characteristic Name	UUID	Properties	Data Format	Description
CPU Usage	<code>...57-da27...</code>	Read, Notify	UTF-8 String (<code>%</code>)	Diagnostic CPU load
CPU Temp	<code>...58-da27...</code>	Read, Notify	UTF-8 String (<code>°C</code>)	Diagnostic core temperature
Battery Level	<code>...59-da27...</code>	Read, Notify	UTF-8 String (<code>%</code>)	Battery state of charge
Soil pH	<code>...60-da27...</code>	Read, Notify	UTF-8 String	Soil pH level (0.0 to 14.0)
Soil Moisture	<code>...61-da27...</code>	Read, Notify	UTF-8 String (<code>%</code>)	Moisture percentage
Soil Temp	<code>...62-da27...</code>	Read, Notify	UTF-8 String (<code>°C</code>)	Soil temperature
Soil EC	<code>...63-da27...</code>	Read, Notify	UTF-8 String	Electrical Conductivity (μ S/cm)
Nitrogen (N)	<code>...64-da27...</code>	Read, Notify	UTF-8 String	Nitrogen level (mg/kg)
Phosphorus (P)	<code>...65-da27...</code>	Read, Notify	UTF-8 String	Phosphorus level (mg/kg)
Potassium (K)	<code>...66-da27...</code>	Read, Notify	UTF-8 String	Potassium level (mg/kg)

Scan Trigger	...67-da27...	Write	Byte (0x01 or 0x02)	Trigger for online/strato scans
ML Output	...68-da27...	Read, Notify	JSON String	JSON output of crop recommendations & rehab

Connection Handshake

1. **Advertising:** The server advertises its service UUID.
2. **Periodic Diagnostic Loop:** Every 2 seconds, the server fetches CPU usage, core temperature, and battery capacity. It updates their respective characteristics and pushes live updates to the Nextion display.
3. **Connection Monitoring:** The server runs hcitool con to check if a BLE connection is active, updating the Nextion's status bar to either **"App Connected"** (green) or **"Waiting for App"** (gray).

4. Data Acquisition Flow (Scanning)

Scanning can be initiated in three ways:

1. **Online Soil Scan** (BLE write 0x01 to SCAN_TRIGGER_UUID)
2. **Strato Scan** (BLE write 0x02 to SCAN_TRIGGER_UUID)
3. **Offline Scan** (Physical GPIO11 Button Press)

Step-by-Step Acquisition Sequence:

```

sequenceDiagram
    autonumber
    actor User/App
    participant Server as BLE Server (Pi)
    participant Display as Nextion Display
    participant Buzzer as Piezo Buzzer
    participant Sensor as RS485 Soil Sensor
    participant ML as ML Model Bundle
    participant DB as SQLite DB

    User/App->>Server: Trigger Scan (Button Press / BLE Write)
    Server->>Display: Transition to scanning page
    Server->>Buzzer: Play start beep
    Note over Server, Sensor: Acquisition Loop (10 iterations, 2s interval)
    loop 10 Times
        Server->>Buzzer: Play short scan tick
        Server->>Sensor: Request holding registers 0x0000 - 0x0006
        Sensor-->>Server: Return raw registers
    end
    Server->>Buzzer: Play 3-beep finish melody
    Server->>Server: Run Pre-Flight Parameter Validation
    alt Validation Failed (Error / Zero Moisture / Zero NPK)
        Server->>Server: Bypass Model, output forced Rehab JSON
    else Validation Passed
        Server->>ML: Run Random Forest Classifier (RFC)
        ML-->>Server: Return Top 5 Crop probabilities & suitability tiers
    end
    Server->>DB: Log scan data
    Server->>User/App: Notify characteristics (N, P, K, pH, EC, Temp, Moisture, ML JSON)
    Server->>Display: Display results/scan done page

```

5. Parameter Validation & Rehabilitation Logic

Before passing sensor data to the Random Forest model, the server executes **Pre-Flight Validation Rules**. If any rule fails, the system bypasses standard ML crop recommendation, categorizes the soil, and forces a specific rehabilitation state.

Validation Branches (Pre-Flight Rules)

Path A: Empty Soil Reading (Sensor Not Inserted)

- **Trigger:** Nitrogen, Phosphorus, Potassium, EC, and Moisture are **all exactly** `>0.0`.
- **Action:** Bypasses the ML model entirely.
- **Output JSON:**

```
{
  "crops": [],
  "has_crop_match": false,
  "soil_type": "any",
  "ml_flags": ["No nutrients detected"],
  "ml_subtext": "No soil readings were detected. Make sure the sensor probe is fully inserted into the soil...",
  "ml_deficiencies": {},
  "rehab": []
}
```

Path B: Moisture is Zero (Dry Soil)

- **Trigger:** Soil moisture is **exactly** `<font face="Courier" color="#c7254e" size="9">0.0</font>` (and NPK are not all zero).
- **Action:** Bypasses ML crop recommendation to prevent inaccurate predictions due to dry conditions.
- **Forced Rehab:** `<font face="Courier" color="#c7254e" size="9">WATER</font>`. If any NPK parameters are also `<font face="Courier" color="#c7254e" size="9">0.0</font>`, their corresponding organic treatments (`<font face="Courier" color="#c7254e" size="9">FPJ</font>`, `<font face="Courier" color="#c7254e" size="9">CalPhos</font>`, `<font face="Courier" color="#c7254e" size="9">FFJ</font>`) are added.
- **Output JSON:**

```
{
  "crops": [],
  "has_crop_match": false,
  "soil_type": "any",
  "ml_flags": ["Dry Soil", "Nutrient deficiency detected"],
  "ml_subtext": "Moisture level is zero. The soil must be watered before a reliable crop recommendation can be made.",
  "ml_deficiencies": { "N": 0 },
  "rehab": ["WATER", "FPJ"]
}
```

Path C: Nutrient Gap (Any NPK is Zero)

- **Trigger:** Moisture is non-zero, but **at least one** NPK parameter is **exactly** `<font face="Courier" color="#c7254e" size="9">0.0</font>`.
- **Action:** Bypasses ML crop recommendation.
- **Forced Rehab:** `<font face="Courier" color="#c7254e" size="9">IMO</font>` (Indigenous Microorganisms) + `<font face="Courier" color="#c7254e" size="9">WATER</font>` + specific organic nutrients corresponding to the zero values:
 - Nitrogen (N = 0): Add `<font face="Courier" color="#c7254e" size="9">FPJ</font>` (Fermented Plant Juice)
 - Phosphorus (P = 0): Add `<font face="Courier" color="#c7254e" size="9">CalPhos</font>` (Calcium Phosphate)
 - Potassium (K = 0): Add `<font face="Courier" color="#c7254e" size="9">FFJ</font>` (Fermented Fruit Juice)
- **Output JSON:**

```
{
  "crops": [],
  "has_crop_match": false,
  "soil_type": "any",
  "ml_flags": ["Nutrient deficiency detected"],
  "ml_subtext": "One or more NPK values are zero. Soil rehabilitation is required before a reliable crop recom",
  "ml_deficiencies": { "N": 0, "K": 0 },
  "rehab": ["IMO", "WATER", "FPJ", "FFJ"]
}
```

Path D: Hard Sensor Error

- **Trigger:** Physically impossible values, specifically **pH** < 3.0 or **temperature** < 5.0°C.
- **Action:** Bypasses ML crop recommendation.
- **Forced Rehab:** **WATER** + **IMO** with a warning subtext indicating physical sensor mismatch.

6. Machine Learning Crop Recommendations

If the soil readings pass all pre-flight checks, the system proceeds to the **Machine Learning Prediction Engine**.

Model Inputs

The features vector contains 8 encoded attributes structured as follows:

$\text{Features} = \begin{bmatrix} N & P & K & \text{Temp} & \text{Moisture} & \text{pH} & \text{EC} & \text{Soil Type Encoded} \end{bmatrix}$

- *Note:* **Soil Type Encoded** is transformed from the string **'any'** using the trained **le_soil** label encoder.

Inference & Suitability Ranking

1. **Probability Output:** The model calculates probability vectors across all crop categories:

$\text{probabilities} = \text{rfc.predict_proba}(\text{features})$

2. **Ranking:** The top 5 crops with the highest probability are extracted.
3. **Suitability Mapping:** Each crop is mapped to a suitability tier based on its rank:

- **Rank 1 & 2:** **OPTIMAL**
- **Rank 3:** **HIGHLY SUITABLE**
- **Rank 4 & 5:** **SUITABLE**

7. Soil Health Flags & Organic Treatments

Even when crop recommendations are generated, the system checks the parameters against optimal ranges in `thresholds` (defined within the PKL bundle) to detect deficiencies and assign natural organic remedies.

Parameter Threshold Audits

Parameter	Audit Logic	Flag Triggered	Target Organic Treatment
Nitrogen (N)	$N < N_{\min}$	Deficiency	FPJ (Fermented Plant Juice)
Phosphorus (P)	$P < P_{\min}$	Deficiency	CalPhos (Calcium Phosphate)
Potassium (K)	$K < K_{\min}$	Deficiency	FFJ (Fermented Fruit Juice)
pH	$pH < pH_{\text{opt_low}}$	<code>High Acidity</code>	LABS (Lactic Acid Bacteria Serum)
pH	$pH > pH_{\text{opt_high}}$	<code>High Alkalinity</code>	LABS (Lactic Acid Bacteria Serum)
EC	$EC < EC_{\text{opt_low}}$	<code>Low EC</code>	LABS (Lactic Acid Bacteria Serum)
EC	$EC > EC_{\text{opt_high}}$	<code>High EC</code>	LABS (Lactic Acid Bacteria Serum)
Moisture	$\text{Moisture} < M_{\text{opt_low}}$	<code>Dry Soil</code>	WATER
Moisture	$\text{Moisture} > M_{\text{opt_high}}$	<code>Waterlogged Soil</code>	— (No additions)

- If no deficiencies or flags are raised, the system defaults to recommending **IMO** (Indigenous Microorganisms) to sustain soil biology.

8. Data Transmission to the App

Once inference and audits are completed:

1. **Raw Parameter Updates:** Individual characteristics (`SOIL_PH_UUID`, `NITROGEN_UUID`, etc.) are updated with UTF-8 strings.
2. **Payload Serialization:** The ML classification and health audit result is serialized into a single JSON object.
3. **GATT Notification:** The JSON payload is written to the `ML_OUTPUT_UUID` characteristic.

4. **App Display:** The BLE stack triggers notification callbacks. The connected mobile application receives the JSON, parses the values, and renders:

- **Crop recommendations** sorted by suitability tiers.
- **Soil parameters** compared with optimal bounds.
- **A checklist of organic solutions** (FPJ, FFJ, CalPhos, LABS, IMO, WATER) with visual indicators highlighting specific soil deficiencies.